

Fase 3 — Semana 2: pipeline de preprocesamiento en clases

Grupo 4 · MCDI500 · Encuesta Nacional de Salud 2016-2017

En la Sumativa 1 dejamos listo un conjunto de 5.511 personas para estudiar cómo se asocian edad, sexo, escolaridad, ingreso y zona con cinco indicadores de riesgo cardiovascular: hipertensión, diabetes, colesterol alto, índice de masa corporal y actividad física. Ese preprocesamiento vivía en funciones sueltas dentro de un notebook.

En esta entrega reescribimos esos mismos pasos como clases que comparten una interfaz común. El criterio de éxito es concreto: el pipeline con clases debe entregar exactamente el mismo conjunto que guardamos en la Fase 2.

Sección	Contenido
1	Configuración y carga del conjunto elegible F1-F2
2	Pipeline de preprocesamiento en clases
3	Validación: casos normal, límite y excepción
4	Codificación de variables categóricas
5	Escalamiento de variables numéricas
6	Evaluación de eficiencia
7	Patrón de diseño Strategy aplicado a la imputación
8	Arquitectura y conclusiones

1. Configuración y carga del conjunto elegible F1-F2

Partimos del archivo filtrado por ponderador en la Fase 2, antes de la limpieza. Así las clases tienen que reproducir todo el preprocesamiento, y el resultado se puede comparar con el conjunto final guardado en esa fase. Las columnas se agrupan según su rol en el estudio: predictoras sociodemográficas, indicadores de riesgo y variables del diseño muestral.

```
In [1]: RUTA_DATOS = "data/processed/ens_variables_f1f2.xlsx"
        COLUMNA_ID = "IdEncuesta"

        # Predictoras sociodemográficas
        COLUMNAS_PREDICTORAS_CONTINUAS = ["Edad", "anos_estudio_MINSAL_1", "as27"]
        COLUMNAS_PREDICTORAS_NOMINALES = ["Sexo", "Zona"]
        COLUMNAS_PREDICTORAS_ORDINALES = ["as28"]
```

```

# Indicadores de riesgo cardiovascular (se analizan por separado)
COLUMNAS_RESULTADO_BINARIAS = ["HTA"]
COLUMNAS_RESULTADO_NOMINALES = ["di3", "dis2"]
COLUMNAS_RESULTADO_ORDINALES = ["GPAQ"]
COLUMNAS_RESULTADO_CONTINUAS = ["IMC"]

# Diseño muestral: se conservan sin transformar
COLUMNAS_DISEÑO_MUESTRAL = ["Fexp_F1F2p_Corr", "Conglomerado", "Estrato"]

SEMILLA = 2026

COLUMNAS_ESPERADAS = (
    [COLUMNA_ID]
    + COLUMNAS_PREDICTORAS_CONTINUAS + COLUMNAS_PREDICTORAS_NOMINALES
    + COLUMNAS_PREDICTORAS_ORDINALES + COLUMNAS_RESULTADO_BINARIAS
    + COLUMNAS_RESULTADO_NOMINALES + COLUMNAS_RESULTADO_ORDINALES
    + COLUMNAS_RESULTADO_CONTINUAS + COLUMNAS_DISEÑO_MUESTRAL
)
print("Columnas declaradas:", len(COLUMNAS_ESPERADAS))

```

Columnas declaradas: 15

Se prepara el entorno: se ubica la raíz del repositorio para poder importar desde `src/`, se importan las funciones de carga y la clase base, y se fija la semilla.

```

In [2]: import sys
        from pathlib import Path

        import numpy as np
        import pandas as pd

        # La raíz se busca aquí porque es la que permite importar src/;
        # por eso no puede venir desde el propio src/carga.py.
        RAIZ = next((p for p in [Path.cwd(), *Path.cwd().parents] if (p / ".git").exists()), None)
        if RAIZ is None:
            raise FileNotFoundError("No se encontró la raíz del repositorio (.git).")
        sys.path.append(str(RAIZ))

        from src.carga import cargar_conjunto, perfilar
        from src.transformador import Transformador

        np.random.seed(SEMILLA)
        print("pandas", pd.__version__, "· NumPy", np.__version__, "· semilla", SEMILLA)

```

pandas 3.0.5 · NumPy 2.5.3 · semilla 2026

Se carga el conjunto elegible con `cargar_conjunto` y se perfila con `perfilar`. La tabla muestra solo las columnas que tienen valores nulos.

```

In [3]: datos = cargar_conjunto(RAIZ / RUTA_DATOS, COLUMNAS_ESPERADAS)
        perfil = perfilar(datos)
        perfil[perfil["nulos"] > 0]

```

Conjunto leído: 5520 filas x 16 columnas

Columnas no usadas en el análisis: ['FechaInicioF1']

Out [3]:

	tipo	nulos	pct_nulos
HTA	float64	9	0.16
IMC	float64	37	0.67
anos_estudio_MINSAL_1	float64	47	0.85
GPAQ	float64	196	3.55
as27	float64	995	18.03

El conjunto tiene 5.520 personas. Los nulos se concentran en `as27` (995), `GPAQ` (196), `anos_estudio_MINSAL_1` (47), `IMC` (37) y `HTA` (9). La no respuesta de `as28` no aparece aquí porque está codificada como -9999: el pipeline debe convertirla en nulo antes de imputar.

2. Pipeline de preprocesamiento en clases

Cada paso de limpieza de la Fase 2 se reescribe como una subclase de `Transformador` (`src/transformador.py`). La clase base fija el orden de uso: `ajustar()` calcula los parámetros con el conjunto de referencia y `transformar()` los aplica sobre una copia, sin volver a calcularlos. Cada subclase solo define qué calcula (`aprender()`) y cómo lo usa (`aplicar()`). Los pasos concretos de imputación, codificación y escalamiento se incorporan en las subsecciones siguientes.

2.1 Preparar los datos antes de imputar

Antes de rellenar cualquier hueco hay que resolver dos cosas que en la Fase 2 se hicieron a mano.

La primera son los códigos de no respuesta. En `as28` (tramo de ingreso del hogar) hay 818 personas con el valor -9999, que pandas trata como un ingreso más.

`MarcadorNoRespuesta` los convierte en nulos, pero antes deja anotado en `as28_no_responde` quién no respondió: no declarar el ingreso puede tener un significado propio y no conviene perderlo.

La segunda son las 9 personas sin diagnóstico de hipertensión (`HTA`). Un diagnóstico no se puede estimar sin afirmar algo que nadie declaró, así que

`EliminadorFilasNulas` las quita en lugar de rellenarlas.

Las dos clases heredan de `Transformador` : solo definen qué calculan y cómo lo aplican. El orden de uso (ajustar antes de transformar) lo controla la clase base.

```
In [4]: from src.imputadores import (
        MarcadorNoRespuesta, EliminadorFilasNulas, ImputadorFlexible,
        PorMedia, PorMediana, PorModa, PorMedianaDeTramo, comparar_estrategias,
```

```

)

marcador = MarcadorNoRespuesta("as28")
marcado = marcador.ajustar_transformar(datos)
print(marcador)
print("Personas con código -9999 en as28 (antes):", int((datos["as28"] == -9999).sum()))
print("Nulos en as28 (después):", int(marcado["as28"].isna().sum()))
print("Marcadas en as28_no_responde:", int(marcado["as28_no_responde"].sum()))

eliminador = EliminadorFilasNulas("HTA")
sin_hta = eliminador.ajustar_transformar(marcado)
print(eliminador)
print(f"Filas: {len(marcado)} -> {len(sin_hta)} ({len(marcado) - len(sin_hta)} eliminadas)")
print("as28_no_responde tras eliminar:", int(sin_hta["as28_no_responde"].sum()))

```

```

MarcadorNoRespuesta(as28) [ajustado]
Personas con código -9999 en as28 (antes): 818
Nulos en as28 (después): 818
Marcadas en as28_no_responde: 818
EliminadorFilasNulas(HTA) [ajustado]
Filas: 5520 -> 5511 (9 eliminadas por HTA nulo)
as28_no_responde tras eliminar: 816

```

Se marcaron 818 personas, pero después de quitar las 9 filas sin HTA quedan 816: dos de esas nueve también tenían -9999 en as28.

El orden importa. La bandera se crea antes de borrar los códigos, y las filas se eliminan antes de imputar, porque en la Fase 2 las medianas se calcularon sobre las 5.511 personas restantes. Si se imputara antes, los valores de relleno cambiarían un poco y el resultado ya no coincidiría.

2.2 Imputación: una clase y varias formas de rellenar

ImputadorFlexible recibe la columna y una estrategia, y no sabe rellenar por sí sola: le pide a la estrategia que calcule con qué y que lo aplique. Las decisiones son las de la Fase 2:

Columna	Estrategia	Por qué
IMC , anos_estudio_MINSAL_1	PorMediana	Distribuciones asimétricas
GPAQ	PorModa	Es ordinal (3,6 % de nulos): la moda conserva una categoría real
as27	PorMedianaDeTramo sobre as28	Solo se imputan las 177 personas que sí declararon su tramo; las 816 que no respondieron ninguno de los dos datos quedan sin valor

```
In [5]: pasos_imputacion = [
    ImputadorFlexible("IMC", PorMediana()),
    ImputadorFlexible("anos_estudio_MINSAL_1", PorMediana()),
    ImputadorFlexible("as27", PorMedianaDeTramo("as28")),
    ImputadorFlexible("GPAQ", PorModa()),
]

imputado = sin_hta
for paso in pasos_imputacion:
    nulos_antes = int(imputado[paso.columna].isna().sum())
    imputado = paso.ajustar_transformar(imputado) # misma llamada para todos
    nulos_despues = int(imputado[paso.columna].isna().sum())
    print(f"{paso.nombre:<40} nulos: {nulos_antes:>4} -> {nulos_despues}")

n_imputadas = int(imputado["as27_imputado"].sum())
n_sin_dato = int(imputado["as27"].isna().sum())
print("as27 imputadas por tramo:", n_imputadas)
print("as27 que quedan sin dato:", n_sin_dato)

assert n_imputadas == 177 and n_sin_dato == 816
assert imputado[["IMC", "anos_estudio_MINSAL_1", "GPAQ"]].isna().sum().sum() == 0
assert len(imputado) == len(sin_hta), "La imputación no debe cambiar el número de filas"
print("Verificado: los conteos coinciden con los de la Fase 2.")
```

```
ImputadorFlexible(IMC) nulos: 37 -> 0
ImputadorFlexible(anos_estudio_MINSAL_1) nulos: 47 -> 0
ImputadorFlexible(as27) nulos: 993 -> 816
ImputadorFlexible(GPAQ) nulos: 196 -> 0
as27 imputadas por tramo: 177
as27 que quedan sin dato: 816
Verificado: los conteos coinciden con los de la Fase 2.
```

Herencia, polimorfismo y encapsulamiento se ven así en este código:

- **Herencia:** las clases MarcadorNoRespuesta, EliminatorFilasNulas, ImputadorFlexible, CodificadorOneHot, EscaladorEstandar, EliminatorColumna y ConvertidorEntero heredan de la clase base Transformador. De esta forma reutilizan el manejo común del estado y el ciclo ajustar() → transformar(), mientras cada subclase implementa únicamente su comportamiento específico mediante aprender() y aplicar().
- **Polimorfismo:** todos los pasos se usan con la misma llamada, `ajustar_transformar`, y `ImputadorFlexible` pide `calcular` y `rellenar` a la estrategia sin saber si es media, mediana, moda o por tramo.
- **Encapsulamiento:** lo aprendido queda guardado en `_parametros`; desde fuera solo se obtiene una copia, y usar un paso sin ajustarlo produce un error.

2.3 Comprobación del encapsulamiento

Las tres pruebas siguientes muestran el estado protegido. Primero se transforma sin haber ajustado, algo que debe fallar (el error se captura a propósito). Luego se usa el

orden correcto, aprendiendo con una parte de las personas y aplicando a la otra. Por último se intenta reemplazar desde fuera lo que el paso aprendió.

```
In [6]: # 1) Transformar antes de ajustar: debe fallar, con un mensaje claro
paso_nuevo = ImputadorFlexible("IMC", PorMediana())
try:
    paso_nuevo.transformar(datos)
except RuntimeError as error:
    print("RuntimeError:", error)

# 2) Orden correcto: se aprende en entrenamiento y se aplica a prueba
entrenamiento = sin_hta.sample(frac=0.8, random_state=SEMILLA)
prueba = sin_hta.drop(index=entrenamiento.index)

paso = ImputadorFlexible("IMC", PorMediana()).ajustar(entrenamiento)
prueba_lista = paso.transformar(prueba)

print("\nMediana aprendida en entrenamiento :", round(paso.parametros["valor"], 1))
print("Mediana propia del conjunto de prueba:", round(prueba["IMC"].median(), 1))
print("Nulos de IMC en prueba tras imputar :", int(prueba_lista["IMC"].isna().sum()))

# 3) Lo aprendido no se puede reemplazar desde fuera
try:
    paso.parametros = {"valor": -1}
except AttributeError as error:
    print("\nAttributeError:", error)
```

RuntimeError: ImputadorFlexible(IMC): falta llamar a ajustar().

Mediana aprendida en entrenamiento : 28.3
Mediana propia del conjunto de prueba: 28.37
Nulos de IMC en prueba tras imputar : 0

AttributeError: property 'parametros' of 'ImputadorFlexible' object has no setter

Sin la comprobación previa, aplicar un paso antes de ajustarlo fallaría de forma confusa o, peor, seguiría de largo sin avisar. Aquí el problema aparece de inmediato y con una explicación.

Las dos medianas son parecidas, pero no idénticas. Lo importante es cuál se usa: a la prueba se le aplica la del entrenamiento, así sus propios datos no influyen en el relleno, aunque la diferencia sea pequeña.

2.4 Comprobación de la herencia

La herencia también se puede verificar en el propio código: el imputador es a la vez un `ImputadorFlexible` y un `Transformador`, su cadena de herencia muestra de dónde viene, y solo `aprender` y `aplicar` están definidos en la clase hija. El resto de los métodos los aporta la clase base.

```
In [7]: paso = ImputadorFlexible("IMC", PorMediana())

print("¿Es ImputadorFlexible?", isinstance(paso, ImputadorFlexible))
print("¿Es Transformador?      ", isinstance(paso, Transformador))
print("Cadena de herencia      :", [clase.__name__ for clase in ImputadorFlexi

print("\nDónde está definido cada método:")
for metodo in ["ajustar", "transformar", "ajustar_transformar", "aprender",
               origen = "ImputadorFlexible" if metodo in ImputadorFlexible.__dict__ else
               print(f" {metodo:<22}{origen}")
```

```
¿Es ImputadorFlexible? True
¿Es Transformador?      True
Cadena de herencia      : ['ImputadorFlexible', 'Transformador', 'object']
```

```
Dónde está definido cada método:
ajustar                Transformador
transformar            Transformador
ajustar_transformar    Transformador
aprender              ImputadorFlexible
aplicar                ImputadorFlexible
```

3. Validación: casos normal, límite y excepción

Cada clase del pipeline se prueba por separado con tres tipos de caso: normal (uso esperado, con datos comunes), límite (situaciones extremas pero válidas, como una columna sin nulos o un pipeline vacío) y excepción (entradas incorrectas, donde lo correcto es que falle con un mensaje claro). Estas pruebas viven en

`tests/casos_por_clase.py` y se ejecutan aquí mismo mediante `ejecutar_pruebas()`, para dejar la evidencia dentro del notebook y no solo en el archivo de tests.

Las clases de codificación y escalamiento se presentan más adelante, en las secciones 4 y 5. Aquí ya se importan desde `src/` y se prueban junto con las demás, de modo que esta sección valida el pipeline completo antes de detallar esos pasos.

```
In [8]: from tests.casos_por_clase import ejecutar_pruebas

informe = ejecutar_pruebas(estrico=False)
print(informe["estado"].value_counts())
informe
```

```
59 pruebas: 59 OK, 0 con problemas
estado
OK          59
Name: count, dtype: int64
```

Out [8] :

	clase	escenario	prueba	estado	
0	Transformador	normal	Ajustar devuelve el mismo paso, guarda lo apre...	OK	
1	Transformador	limite	parametros entrega una copia: modificarla no a...	OK	
2	Transformador	limite	transformar trabaja sobre una copia y deja int...	OK	
3	Transformador	excepcion	transformar antes de ajustar lanza RuntimeError.	OK	RuntimeError: _PasoD falt
4	Transformador	excepcion	Ajustar con una columna que no existe lanza Ke...	OK	KeyError: "_PasoDob 'otr
5	Transformador	excepcion	La clase base no se usa sola: sin aprender() l...	OK	NotImplem Transformador no i
6	Transformador	excepcion	parametros no se puede reasignar desde fuera (...)	OK	AttributeErrc 'parametros
7	MarcadorNoRespuesta	normal	Pasa los códigos -8888 y -9999 a nulo y anota ...	OK	
8	MarcadorNoRespuesta	limite	Sin códigos presentes no cambia ningún valor y...	OK	
9	MarcadorNoRespuesta	limite	Un nulo que ya venía en los datos no cuenta co...	OK	
10	MarcadorNoRespuesta	excepcion	Una columna que no existe lanza KeyError.	OK	"MarcadorNoRespi
11	EliminadorFilasNulas	normal	Quita las filas con nulo en la columna y renum...	OK	
12	EliminadorFilasNulas	limite	Sin nulos en la columna devuelve los mismos da...	OK	
13	EliminadorFilasNulas	limite	Si todas las filas son nulas queda un DataFram...	OK	
14	EliminadorFilasNulas	limite	Los nulos de otras columnas no provocan que se...	OK	

	clase	escenario	prueba	estado	
15	EliminadorFilasNulas	excepcion	Una columna que no existe lanza KeyError.	OK	"EliminadorFilasNulas"
16	PorMedia	normal	Rellena los nulos con la media de los datos pr...	OK	
17	PorMedia	limite	Con un único dato válido, la media es ese dato.	OK	
18	PorMedia	excepcion	Una columna sin ningún dato lanza ValueError.	OK	ValueError: 'x' no para
19	PorMediana	normal	Rellena los nulos con la mediana de los datos ...	OK	
20	PorMediana	limite	Con cantidad par de datos, la mediana es el pr...	OK	
21	PorMediana	excepcion	Una columna sin ningún dato lanza ValueError.	OK	ValueError: 'x' no para
22	PorModa	normal	Rellena los nulos con el valor más frecuente.	OK	
23	PorModa	limite	Con empate entre valores, se elige el menor (i...	OK	
24	PorModa	excepcion	Una columna sin ningún dato lanza ValueError.	OK	ValueError: 'x' no para
25	PorMedianaDeTramo	normal	Rellena con la mediana de su propio tramo y ma...	OK	
26	PorMedianaDeTramo	limite	Una fila sin tramo queda nula y no se marca co...	OK	
27	PorMedianaDeTramo	limite	Si un tramo no tiene ningún monto, sus filas q...	OK	
28	PorMedianaDeTramo	excepcion	Si la columna de tramo no existe lanza KeyError.	OK	KeyError: "'tramo' no existe"
29	ImputadorFlexible	normal	Sin estrategia usa la mediana y la deja guarda...	OK	

	clase	escenario	prueba	estado	
30	ImputadorFlexible	normal	Con cualquier estrategia se usa la misma llama...	OK	
31	ImputadorFlexible	limite	Una columna sin ningún nulo no se altera.	OK	
32	ImputadorFlexible	limite	Aprende en entrenamiento y aplica a prueba con...	OK	
33	ImputadorFlexible	excepcion	Una estrategia que no es EstrategiaImputacion ...	OK	TypeError: La estrateg una
34	ImputadorFlexible	excepcion	Ajustar sobre una columna sin ningún dato lanz...	OK	ValueError: 'x' no para
35	EliminadorColumna	normal	Quita la columna indicada y deja intactas las ...	OK	
36	EliminadorColumna	limite	Si es la única columna, queda una tabla sin co...	OK	
37	EliminadorColumna	excepcion	Una columna que no existe lanza KeyError.	OK	"EliminadorColumna(lc
38	ConvertidorEntero	normal	Convierte una columna decimal sin nulos a ente...	OK	
39	ConvertidorEntero	limite	Una columna que ya es entera queda igual.	OK	
40	ConvertidorEntero	excepcion	Una columna con nulos lanza ValueError al ajus...	OK	ConvertidorEnte
41	CodificadorOneHot	normal	Crea una columna 0/1 por categoría, quita la o...	OK	
42	CodificadorOneHot	limite	Con una sola categoría presente se generan igu...	OK	
43	CodificadorOneHot	limite	Ajustado con un conjunto, a otro con menos cat...	OK	
44	CodificadorOneHot	excepcion	Un código que no está en el libro de	OK	CodificadorOneHc

	clase	escenario	prueba	estado	
			códigos l...		
45	CodificadorOneHot	excepcion	Un código nuevo que aparece solo al transforma...	OK	CodificadorOneHo
46	CodificadorOneHot	excepcion	Una columna sin categorías definidas lanza Val...	OK	CodificadorOneHo
47	EscaladorEstandar	normal	Deja la columna con media 0 y desviación 1, y ...	OK	
48	EscaladorEstandar	limite	Los nulos se conservan y no entran en el cálcu...	OK	
49	EscaladorEstandar	limite	Aprende en entrenamiento y aplica a prueba la ...	OK	
50	EscaladorEstandar	excepcion	Una columna constante (desviación cero) lanza ...	OK	EscaladorEstandi
51	EscaladorEstandar	excepcion	Una columna sin ningún dato lanza ValueError.	OK	EscaladorEstandi
52	Pipeline	normal	Encadena los pasos en orden: marca, elimina fi...	OK	
53	Pipeline	limite	Un pipeline sin pasos devuelve los datos sin c...	OK	
54	Pipeline	limite	Agregar un paso después de ajustar obliga a aj...	OK	RuntimeError: Pij llamar
55	Pipeline	limite	Ajustar y transformar dejan intacto el DataFra...	OK	
56	Pipeline	excepcion	Agregar algo que no es un Transformador lanza ...	OK	TypeError: Pipeline : Transf
57	Pipeline	excepcion	transformar antes de ajustar lanza RuntimeError.	OK	RuntimeError: Pij llamar
58	Pipeline	excepcion	Un paso que apunta a una columna inexistente d...	OK	"ImputadorFlexible

Las 59 pruebas cubren las 13 clases del pipeline (`Transformador` , los siete pasos de `imputadores.py` , los cuatro de `transformadores.py` y `Pipeline`), repartidas en los tres escenarios. Los 13 casos restantes de la suite completa (72 en total, ver `tests/test_transformadores.py`) prueban específicamente `CodificadorOneHot` y `EscaladorEstandar` con `pytest` y no pasan por `ejecutar_pruebas()` , por lo que no aparecen en esta tabla.

4. Codificación de variables categóricas

Como parte del pipeline de preprocesamiento orientado a objetos, se incorpora la clase `CodificadorOneHot` , que hereda de la clase base `Transformador` .

Su objetivo es transformar las variables categóricas seleccionadas de la ENS en columnas binarias mediante **One-Hot Encoding**, manteniendo nombres semánticos asociados a las categorías originales.

Las variables consideradas son:

- `Sexo`
- `Zona`
- `di3`
- `dis2`

La implementación mantiene la misma interfaz utilizada por los demás componentes del pipeline: `ajustar()` , `transformar()` y `ajustar_transformar()` .

```
In [9]: from src.transformadores import (  
        CodificadorOneHot,  
        EscaladorEstandar,  
        EliminadorColumna,  
        ConvertidorEntero,  
        )
```

4.1 Categorías utilizadas

La codificación utiliza las categorías definidas para las variables seleccionadas de la ENS.

Además de realizar la transformación, `CodificadorOneHot` valida durante el ajuste que los códigos observados correspondan a categorías conocidas. De esta manera, un código no contemplado genera una excepción en lugar de ser procesado silenciosamente.

```
In [10]: variables_categoricas = ["Sexo", "Zona", "di3", "dis2"]  
  
        for columna in variables_categoricas:
```

```
print(f"\n--- {columna} ---")
print(
    imputado[columna]
    .value_counts(dropna=False)
    .sort_index()
)
```

--- Sexo ---

Sexo

1 2017

2 3494

Name: count, dtype: int64

--- Zona ---

Zona

1 4638

2 873

Name: count, dtype: int64

--- di3 ---

di3

1 815

2 4645

3 51

Name: count, dtype: int64

--- dis2 ---

dis2

1 767

2 643

3 3830

4 271

Name: count, dtype: int64

Definición de categorías

Las categorías utilizadas por `CodificadorOneHot` se definen explícitamente a partir del libro de códigos de la ENS 2016–2017 y no se infieren únicamente desde los valores observados en cada muestra.

Esta decisión permite mantener un esquema de salida estable. Por ejemplo, si una muestra contiene únicamente el código `1` para `Sexo`, el transformador conserva igualmente las columnas correspondientes a todas las categorías válidas definidas para esa variable.

Además, tanto durante el ajuste como durante la transformación se valida que los códigos observados pertenezcan al conjunto de categorías permitidas, evitando incorporar silenciosamente valores no reconocidos.

4.2 Aplicación y validación de `CodificadorOneHot`

Cada variable categórica se transforma utilizando una instancia independiente de `CodificadorOneHot`.

Durante `ajustar()`, el transformador valida los códigos presentes y almacena las categorías correspondientes. Posteriormente, `transformar()` genera una columna binaria por categoría y elimina la variable categórica original.

El proceso se aplica secuencialmente para mantener la lógica común definida por la clase base `Transformador`.

```
In [11]: df_codificado = imputado.copy()

codificadores = {}

for columna in variables_categoricas:
    codificador = CodificadorOneHot(columna)

    df_codificado = codificador.ajustar_transformar(
        df_codificado
    )

    codificadores[columna] = codificador

print(
    "Dimensiones antes de codificar:",
    imputado.shape
)

print(
    "Dimensiones después de codificar:",
    df_codificado.shape
)
```

Dimensiones antes de codificar: (5511, 18)

Dimensiones después de codificar: (5511, 25)

Se listan las columnas que generó la codificación, para confirmar que corresponden a las categorías de `Sexo`, `Zona`, `di3` y `dis2`.

```
In [12]: columnas_onehot = [
    columna
    for columna in df_codificado.columns
    if columna.startswith(("Sexo_", "Zona_", "di3_", "dis2_"))
]

print("Columnas generadas:")
for columna in columnas_onehot:
    print("-", columna)
```

Columnas generadas:

- Sexo_Hombre
- Sexo_Mujer
- Zona_Urbano
- Zona_Rural
- di3_Si
- di3_No
- di3_No_recuerda
- dis2_Si_una_vez
- dis2_Si_mas_de_una_vez
- dis2_Nunca
- dis2_No_recuerda

Se revisan los valores únicos de cada columna generada: deben ser solo 0 y 1.

```
In [13]: print("Valores únicos por columna:\n")

for columna in columnas_onehot:
    print(
        columna,
        sorted(df_codificado[columna].unique())
    )
```

Valores únicos por columna:

```
Sexo_Hombre [np.int64(0), np.int64(1)]
Sexo_Mujer [np.int64(0), np.int64(1)]
Zona_Urbano [np.int64(0), np.int64(1)]
Zona_Rural [np.int64(0), np.int64(1)]
di3_Si [np.int64(0), np.int64(1)]
di3_No [np.int64(0), np.int64(1)]
di3_No_recuerda [np.int64(0), np.int64(1)]
dis2_Si_una_vez [np.int64(0), np.int64(1)]
dis2_Si_mas_de_una_vez [np.int64(0), np.int64(1)]
dis2_Nunca [np.int64(0), np.int64(1)]
dis2_No_recuerda [np.int64(0), np.int64(1)]
```

4.3 Validación de la codificación

La transformación generó correctamente las variables binarias asociadas a las categorías de `Sexo`, `Zona`, `di3` y `dis2`.

Las columnas resultantes contienen exclusivamente valores `0` y `1`, mientras que las variables categóricas originales son retiradas del conjunto transformado.

La implementación permite además conservar la semántica de las categorías mediante nombres descriptivos, evitando utilizar únicamente los códigos numéricos originales de la ENS.

Las validaciones unitarias y los casos de excepción de este componente se encuentran implementados separadamente en `tests/test_transformadores.py`.

5. Escalamiento de variables numéricas

Como siguiente etapa del pipeline se incorpora `EscaladorEstandar`, una subclase de `Transformador` destinada a estandarizar variables numéricas.

Para una observación (x), la transformación aplicada corresponde a:

$$[z = \frac{x - \mu}{\sigma}]$$

donde:

- (x) es el valor original;
- (μ) es la media calculada durante el ajuste;
- (σ) es la desviación estándar calculada durante el ajuste.

La separación entre `ajustar()` y `transformar()` permite almacenar los parámetros aprendidos y reutilizarlos posteriormente sobre nuevos datos.

5.1 Validación de `EscaladorEstandar`

Antes de aplicar una transformación numérica se verifica que las variables seleccionadas no contengan valores faltantes.

Esta comprobación es relevante porque el escalamiento corresponde a una etapa posterior al tratamiento de valores nulos dentro del pipeline de preprocesamiento.

```
In [14]: print("Estado de as27 después de la etapa de imputación:")
print("Registros totales:", len(imputado))
print("Valores disponibles:", imputado["as27"].notna().sum())
print("Valores nulos:", imputado["as27"].isna().sum())

print("\nResumen estadístico de as27:")
print(imputado["as27"].describe())
```

```
Estado de as27 después de la etapa de imputación:
Registros totales: 5511
Valores disponibles: 4695
Valores nulos: 816
```

```
Resumen estadístico de as27:
count    4.695000e+03
mean     4.689790e+05
std       6.202431e+05
min       2.000000e+04
25%       2.000000e+05
50%       3.000000e+05
75%       5.000000e+05
max       1.600000e+07
Name: as27, dtype: float64
```

Se listan las columnas del conjunto relacionadas con el ingreso (`as27` , `as28` y sus banderas), para confirmar en qué estado llegan a la etapa de escalamiento.

```
In [15]: columnas_ingreso = [
        columna for columna in imputado.columns
        if "as27" in columna.lower() or "as28" in columna.lower()
    ]

print("Columnas relacionadas con ingreso:")
print(columnas_ingreso)
```

```
Columnas relacionadas con ingreso:
['as27', 'as28', 'as28_no_responde', 'as27_imputado']
```

5.2 Aplicación de EscaladorEstandar

El escalamiento se aplica después de la etapa de imputación.

`Edad` e `IMC` pueden escalarse sobre la totalidad de los registros disponibles. En el caso de `as27` , se conservan los valores faltantes definidos por la estrategia de imputación de la Fase 2.

La estrategia `PorMedianaDeTramo` imputa únicamente los casos en que existe información del tramo de ingreso (`as28`). Los participantes que no entregaron información suficiente mantienen `as27` como valor faltante, evitando introducir ingresos artificiales.

Por lo tanto, el escalamiento no modifica la política de tratamiento de valores faltantes definida previamente.

```
In [16]: df_escalado = imputado.copy()

variables_numericas = ["Edad", "IMC", "as27"]

escaladores = {}

for columna in variables_numericas:
    escalador = EscaladorEstandar(columna)

    df_escalado = escalador.ajustar_transformar(
        df_escalado
    )

    escaladores[columna] = escalador

print("Dimensiones:", df_escalado.shape)

print("\nNulos después del escalamiento:")
print(df_escalado[variables_numericas].isna().sum())
```

Dimensiones: (5511, 18)

Nulos después del escalamiento:

```
Edad      0
IMC       0
as27     816
dtype: int64
```

5.3 Validación estadística del escalamiento

Para verificar el funcionamiento de `EscaladorEstandar`, se comprueba que las variables transformadas presenten una media cercana a 0 y una desviación estándar cercana a 1.

En `as27`, las estadísticas se calculan sobre los valores disponibles. Los 816 valores faltantes definidos por la estrategia de imputación anterior se conservan y no intervienen en el cálculo.

```
In [17]: for columna in variables_numericas:
          media = df_escalado[columna].mean()
          desviacion = df_escalado[columna].std(ddof=0)

          print(f"\n{columna}")
          print("Media:", round(media, 10))
          print("Desviación estándar:", round(desviacion, 10))
```

```
Edad
Media: 0.0
Desviación estándar: 1.0
```

```
IMC
Media: -0.0
Desviación estándar: 1.0
```

```
as27
Media: -0.0
Desviación estándar: 1.0
```

5.4 Resultado del escalamiento

Las tres variables presentan una media aproximadamente igual a 0 y una desviación estándar igual a 1 después de la transformación, confirmando el funcionamiento esperado de `EscaladorEstandar`.

En `as27` se mantienen los 816 valores faltantes provenientes de la etapa anterior. El escalador transforma únicamente los valores disponibles y no modifica la estrategia de imputación definida previamente.

Esta separación de responsabilidades permite que cada componente del pipeline mantenga una función específica: los imputadores gestionan los valores faltantes y el escalador realiza exclusivamente la transformación numérica.

6. Evaluación de eficiencia

Además de validar el funcionamiento de los transformadores, se evalúa su eficiencia computacional mediante comparaciones con implementaciones de referencia.

Se consideran dos dimensiones:

- **Tiempo de ejecución:** permite comparar el costo temporal de cada implementación.
- **Memoria pico:** permite estimar la memoria utilizada durante la ejecución.

Las mediciones se realizan mediante las funciones reutilizables definidas en `src/medicion.py`.

Se comparan:

1. `EscaladorEstandar` frente a `StandardScaler` de scikit-learn.
2. `CodificadorOneHot` frente a `pandas.get_dummies`.

Además de las mediciones empíricas, se analiza la complejidad temporal y espacial de las soluciones.

```
In [18]: from src.medicion import (
        medir_tiempo,
        medir_memoria,
        medir_tiempo_timeit,
    )

    from sklearn.preprocessing import StandardScaler
```

Se definen las dos versiones que se comparan: el escalador propio, `EscaladorEstandar`, y el de scikit-learn, `StandardScaler`. Ambas reciben la misma columna y devuelven un DataFrame.

```
In [19]: def escalar_propio(df, columna):
        escalador = EscaladorEstandar(columna)

        return escalador.ajustar_transformar(
            df[[columna]].copy()
        )

    def escalar_sklearn(df, columna):
        escalador = StandardScaler()

        resultado = df[[columna]].copy()

        resultado[columna] = escalador.fit_transform(
            resultado[[columna]]
        )
```

```
).ravel()

return resultado
```

6.1 Equivalencia de resultados

Antes de comparar eficiencia, se verifica que ambas implementaciones produzcan resultados numéricamente equivalentes.

Esta comprobación evita comparar el rendimiento de algoritmos que realizan transformaciones diferentes.

```
In [20]: datos_edad = imputado[["Edad"]].copy()

resultado_propio = escalar_propio(
    datos_edad,
    "Edad"
)

resultado_sklearn = escalar_sklearn(
    datos_edad,
    "Edad"
)

np.testing.assert_allclose(
    resultado_propio["Edad"].to_numpy(),
    resultado_sklearn["Edad"].to_numpy(),
    rtol=1e-10,
    atol=1e-10,
)

print("Resultados equivalentes: EscaladorEstandar == StandardScaler")
```

Resultados equivalentes: EscaladorEstandar == StandardScaler

Se mide el tiempo de las dos versiones con `timeit`, repitiendo el conjunto 1, 10, 50 y 100 veces, para observar cómo cambia el tiempo cuando aumentan los datos.

```
In [21]: factores = [1, 10, 50, 100]

base_escalamiento = imputado[["Edad"]].copy()

resultados_escalamiento_timeit = []

for factor in factores:
    muestra = pd.concat(
        [base_escalamiento] * factor,
        ignore_index=True
    )

    tiempo_propio = medir_tiempo_timeit(
        escalar_propio,
        muestra,
        "Edad"
    )
```

```

    )

    tiempo_sklearn = medir_tiempo_timeit(
        escalar_sklearn,
        muestra,
        "Edad"
    )

    resultados_escalamiento_timeit.append({
        "factor": factor,
        "filas": len(muestra),
        "propio_s": tiempo_propio,
        "sklearn_s": tiempo_sklearn
    })

resultados_escalamiento_timeit = pd.DataFrame(
    resultados_escalamiento_timeit
)

resultados_escalamiento_timeit

```

Out[21]:

	factor	filas	propio_s	sklearn_s
0	1	5511	0.000421	0.001288
1	10	55110	0.000305	0.000926
2	50	275550	0.001250	0.001933
3	100	551100	0.002493	0.003168

Análisis de resultados

Para evaluar el comportamiento temporal del escalamiento se utilizó `timeit` sobre conjuntos de tamaño creciente, obtenidos mediante la replicación controlada del conjunto de referencia. Se evaluaron tamaños entre 5.511 y 551.100 observaciones.

Ambas implementaciones presentan un incremento del tiempo de ejecución a medida que aumenta el número de registros, comportamiento compatible con una complejidad temporal lineal ($O(n)$), dado que el escalamiento requiere procesar cada observación de la variable.

`EscaladorEstandar` presentó menores tiempos de ejecución que `StandardScaler` en los tamaños evaluados. Sin embargo, esta diferencia no implica una menor complejidad algorítmica, ya que ambas implementaciones son ($O(n)$). Las diferencias observadas pueden estar asociadas a costos constantes y a las validaciones y operaciones internas realizadas por cada implementación.

En consecuencia, la implementación propia se mantiene principalmente por su integración con la arquitectura `Transformador`, su comportamiento controlado dentro

del pipeline y su utilidad pedagógica, y no únicamente por la diferencia de tiempo observada.

6.2 Medición de memoria del escalamiento

Una vez comprobada la equivalencia y evaluado el comportamiento temporal mediante `timeit`, se complementa el análisis mediante la medición de memoria pico.

Para esta medición se utiliza `tracemalloc`, a través de la función reutilizable `medir_memoria` definida en `src/medicion.py`.

El objetivo es comparar la memoria administrada por Python durante la ejecución de `EscaladorEstandar` y `StandardScaler`. Esta medición debe interpretarse como una aproximación a las asignaciones de memoria gestionadas por Python y no como una medición de la memoria total utilizada por el proceso.

```
In [22]: factores = [1, 10, 50, 100]

base_escalamiento = imputado[["Edad"]].copy()

resultados_memoria_escalamiento = []

for factor in factores:
    muestra = pd.concat(
        [base_escalamiento] * factor,
        ignore_index=True
    )

    memoria_propia, _ = medir_memoria(
        escalar_propio,
        muestra,
        "Edad"
    )

    memoria_sklearn, _ = medir_memoria(
        escalar_sklearn,
        muestra,
        "Edad"
    )

    resultados_memoria_escalamiento.append({
        "factor": factor,
        "filas": len(muestra),
        "memoria_propia_kb": memoria_propia / 1024,
        "memoria_sklearn_kb": memoria_sklearn / 1024,
    })

resultados_memoria_escalamiento = pd.DataFrame(
    resultados_memoria_escalamiento
)

resultados_memoria_escalamiento
```

Out [22]:

	factor	filas	memoria_propia_kb	memoria_sklearn_kb
0	1	5511	181.739258	145.881836
1	10	55110	1730.369141	1356.238281
2	50	275550	8618.743164	6737.777344
3	100	551100	17229.368164	13465.166016

6.3 Análisis de eficiencia del escalamiento

La evaluación temporal realizada mediante `timeit` muestra que tanto `EscaladorEstandar` como `StandardScaler` incrementan su tiempo de ejecución a medida que aumenta el número de observaciones. Este comportamiento es consistente con una complejidad temporal lineal ($O(n)$), ya que ambas implementaciones deben procesar las observaciones de la variable para realizar el escalamiento.

En los tamaños evaluados, `EscaladorEstandar` presentó menores tiempos de ejecución que `StandardScaler`. Sin embargo, esta diferencia corresponde al rendimiento experimental observado y no implica una menor complejidad algorítmica, puesto que ambas alternativas presentan un crecimiento compatible con ($O(n)$).

La evaluación de memoria mediante `tracemalloc` complementa el análisis temporal utilizando los mismos factores de crecimiento del conjunto de datos. Esta medición representa la memoria pico administrada por Python durante cada ejecución y no la memoria total utilizada por el proceso.

Considerando la equivalencia numérica previamente verificada, el comportamiento temporal observado y la integración directa con la arquitectura basada en `Transformador`, se mantiene `EscaladorEstandar` como implementación del proyecto. La decisión no se fundamenta únicamente en diferencias de tiempo o memoria, sino también en su integración, control y mantenibilidad dentro del pipeline.

```
In [23]: def codificar_propio(df, columna):
          codificador = CodificadorOneHot(columna)

          return codificador.ajustar_transformar(
              df[[columna]].copy()
          )

def codificar_pandas(df, columna):
    resultado = pd.get_dummies(
        df[[columna]],
        columns=[columna],
        prefix=columna,
        dtype=int
    )
```

```

if columna == "Sexo":
    resultado = resultado.rename(
        columns={
            "Sexo_1": "Sexo_Hombre",
            "Sexo_2": "Sexo_Mujer",
        }
    )

return resultado

```

Antes de comparar tiempos se verifica que `CodificadorOneHot` y `pandas.get_dummies` entregan el mismo resultado sobre `Sexo`.

```

In [24]: datos_sexo = imputado[["Sexo"]].copy()

resultado_propio_ohe = codificar_propio(
    datos_sexo,
    "Sexo"
)

resultado_pandas_ohe = codificar_pandas(
    datos_sexo,
    "Sexo"
)

pd.testing.assert_frame_equal(
    resultado_propio_ohe,
    resultado_pandas_ohe
)

print(
    "Resultados equivalentes: "
    "CodificadorOneHot == pandas.get_dummies"
)

```

Resultados equivalentes: `CodificadorOneHot == pandas.get_dummies`

6.4 Evaluación de eficiencia de `CodificadorOneHot`

Para evaluar el segundo transformador desarrollado, se compara `CodificadorOneHot` con `pandas.get_dummies`.

Primero se verifica que ambas implementaciones generen una representación binaria equivalente. Posteriormente se comparan sus tiempos de ejecución y memoria pico para distintos tamaños de entrada.

6.5 Medición de eficiencia de la codificación

Una vez comprobada la equivalencia entre `CodificadorOneHot` y `pandas.get_dummies`, se evalúa el comportamiento temporal mediante `timeit` y la memoria pico mediante `tracemalloc`.

Para observar el comportamiento frente al crecimiento del conjunto de datos, se utiliza como base la variable `Sexo` del conjunto de referencia y se generan conjuntos de tamaño creciente mediante factores de replicación 1, 10, 50 y 100.

De esta forma, se evalúan conjuntos desde 5.511 hasta 551.100 observaciones. La variable mantiene dos categorías válidas, por lo que el número de categorías (k) permanece constante durante el experimento.

```
In [25]: factores = [1, 10, 50, 100]

base_codificacion = imputado[["Sexo"]].copy()

resultados_codificacion = []

for factor in factores:
    muestra = pd.concat(
        [base_codificacion] * factor,
        ignore_index=True
    )

    # Medición temporal con timeit
    tiempo_propio = medir_tiempo_timeit(
        codificar_propio,
        muestra,
        "Sexo"
    )

    tiempo_pandas = medir_tiempo_timeit(
        codificar_pandas,
        muestra,
        "Sexo"
    )

    # Medición de memoria con tracemalloc
    memoria_propia, _ = medir_memoria(
        codificar_propio,
        muestra,
        "Sexo"
    )

    memoria_pandas, _ = medir_memoria(
        codificar_pandas,
        muestra,
        "Sexo"
    )

    resultados_codificacion.append({
        "factor": factor,
        "filas": len(muestra),
        "tiempo_propio_s": tiempo_propio,
        "tiempo_pandas_s": tiempo_pandas,
        "memoria_propia_kb": memoria_propia / 1024,
        "memoria_pandas_kb": memoria_pandas / 1024,
    })
```

```
resultados_codificacion = pd.DataFrame(
    resultados_codificacion
)

resultados_codificacion
```

Out[25]:

	factor	filas	tiempo_propio_s	tiempo_pandas_s	memoria_propia_kb	memoria_
0	1	5511	0.000454	0.000422	222.421875	
1	10	55110	0.000686	0.000779	2932.562500	2
2	50	275550	0.002027	0.002831	12569.000000	10
3	100	551100	0.003692	0.004417	25130.156250	20

6.6 Análisis de eficiencia de la codificación

Las mediciones realizadas con `timeit` muestran que tanto `CodificadorOneHot` como `pandas.get_dummies` incrementan su tiempo de ejecución a medida que aumenta el número de observaciones.

La implementación propia puede expresarse con una complejidad temporal ($O(n \cdot k)$), donde (n) corresponde al número de observaciones y (k) al número de categorías. En este experimento, `Sexo` mantiene dos categorías válidas, por lo que (k) permanece constante y el crecimiento respecto de (n) se comporta de forma aproximadamente lineal.

Para 5.511 observaciones, `pandas.get_dummies` presentó un tiempo ligeramente menor. A partir de los tamaños mayores evaluados, las diferencias cambian y `CodificadorOneHot` presentó menores tiempos en este experimento. Estas diferencias corresponden al rendimiento empírico observado y no representan una diferencia en el orden de complejidad algorítmica.

La medición con `tracemalloc` muestra un aumento de la memoria pico a medida que crece el conjunto de datos. En los tamaños evaluados, `pandas.get_dummies` presentó un menor consumo de memoria administrada por Python que la implementación propia. Esta medición debe interpretarse como memoria gestionada por Python y no como memoria total del proceso.

Considerando la equivalencia previamente verificada, el comportamiento temporal y espacial observado y los requisitos arquitectónicos del proyecto, se mantiene `CodificadorOneHot`. La decisión se fundamenta principalmente en su integración con `Transformador`, la validación explícita de códigos y la generación de un esquema de categorías controlado y estable.

6.7 Decisión técnica adoptada

Las mediciones realizadas muestran que las implementaciones propias y las alternativas de referencia presentan comportamientos compatibles con un crecimiento lineal respecto del número de observaciones cuando el número de variables o categorías se mantiene constante.

En el escalamiento, `EscaladorEstandar` presentó menores tiempos de ejecución que `StandardScaler` en los tamaños evaluados. En la codificación, `CodificadorOneHot` presentó menores tiempos en los conjuntos de mayor tamaño, mientras que `pandas.get_dummies` mostró un menor consumo de memoria pico administrada por Python.

Por lo tanto, la decisión de mantener `EscaladorEstandar` y `CodificadorOneHot` no se fundamenta únicamente en cuál implementación obtiene el menor tiempo o consumo de memoria en una medición particular. Se consideran también los requisitos de diseño del proyecto.

Las implementaciones propias se integran directamente con la arquitectura basada en `Transformador`, mantienen una interfaz común de ajuste y transformación, permiten validar explícitamente los códigos admitidos y conservan un esquema estable de categorías definido previamente. Estas características favorecen la cohesión, trazabilidad y mantenibilidad del pipeline.

En consecuencia, se adoptan `EscaladorEstandar` y `CodificadorOneHot` como componentes del pipeline de preprocesamiento. Las implementaciones de `scikit-learn` y `pandas` se utilizan como referencias para verificar equivalencia funcional y comparar experimentalmente el comportamiento temporal y espacial.

Síntesis de eficiencia temporal y espacial

Para comparar ambas dimensiones de eficiencia se considera el escenario de mayor tamaño evaluado, correspondiente a 551.100 filas (factor 100).

Operación	Implementación	Tiempo (s)	Memoria pico (KB)
Escalamiento	<code>EscaladorEstandar</code>	0,008468	17.229,38
Escalamiento	<code>StandardScaler</code>	0,009866	13.465,09
One-Hot Encoding	<code>CodificadorOneHot</code>	0,010592	25.130,16
One-Hot Encoding	<code>pandas.get_dummies</code>	0,014671	20.827,30

En el escenario de mayor tamaño, las implementaciones propias presentan menores tiempos de ejecución, mientras que las implementaciones de referencia utilizan menos memoria pico. Por lo tanto, los resultados muestran un compromiso entre eficiencia temporal y espacial, y no una superioridad absoluta de una implementación sobre otra. Tanto en escalamiento como en codificación, el crecimiento observado al aumentar el

número de filas es compatible con una complejidad temporal y espacial lineal ($O(n)$). Las diferencias corresponden principalmente a factores constantes asociados a las operaciones internas de cada implementación.

Se mantienen `EscaladorEstandar` y `CodificadorOneHot` en el pipeline porque, además del rendimiento temporal observado, se integran directamente con la interfaz común `Transformador`, permiten controlar explícitamente las categorías y parámetros aprendidos y mantienen una arquitectura homogénea. La decisión, por tanto, considera conjuntamente tiempo, memoria, mantenibilidad e integración arquitectónica, y no únicamente el menor tiempo de ejecución.

Nota sobre la medición de memoria: `tracemalloc` registra asignaciones de memoria administradas por Python. Por esta razón, los valores obtenidos se utilizan como evidencia comparativa entre implementaciones dentro del mismo entorno de ejecución y no como una medición absoluta de toda la memoria utilizada por el proceso.

6.8 Integración del pipeline de transformación

Los pasos usados y validados por separado en las secciones anteriores (imputación, eliminación de columnas, conversión de tipos, codificación y escalamiento) se encadenan ahora en un único objeto `Pipeline`. El `Pipeline` no conoce el detalle interno de cada paso: solo exige que todos respondan a `ajustar()` y `transformar()`, gracias a que todos heredan de `Transformador`. El proceso parte desde `datos` (el conjunto crudo cargado al inicio del notebook) y aplica los 9 pasos en una sola llamada, reproduciendo el mismo resultado que se obtuvo de forma manual en la Fase 2.

```
In [26]: from src.pipeline import Pipeline

pasos_integracion = (
    [EliminadorColumna(col) for col in ["IdEncuesta", "FechaInicioF1"]]
    + [ConvertidorEntero(col) for col in ["HTA", "GPAQ"]]
    + [CodificadorOneHot(col) for col in variables_categoricas]
    + [EscaladorEstandar(col) for col in variables_numericas]
)

pipeline = Pipeline(pasos_integracion)
df_transformado = pipeline.ajustar_transformar(imputado)

print("Dimensiones iniciales:", imputado.shape)
print("Dimensiones finales:", df_transformado.shape)
```

```
Dimensiones iniciales: (5511, 18)
Dimensiones finales: (5511, 23)
```

Se comprueban con `assert` las propiedades esperadas del resultado del pipeline: mismo número de filas, columnas no analíticas eliminadas, tipos enteros, categorías de `GPAQ`, columnas One-Hot con solo 0 y 1, escalamiento con media 0 y desviación 1, y dimensión final de 5.511×23 .

```
In [27]: # Se mantiene el número de observaciones
assert len(df_transformado) == len(imputado)

# Las variables no analíticas fueron eliminadas
assert "IdEncuesta" not in df_transformado.columns
assert "FechaInicioF1" not in df_transformado.columns

# HTA y GPAQ fueron convertidas a tipo entero
assert pd.api.types.is_integer_dtype(
    df_transformado["HTA"]
)
assert pd.api.types.is_integer_dtype(
    df_transformado["GPAQ"]
)

# GPAQ conserva las categorías ordinales esperadas
assert set(df_transformado["GPAQ"].unique()) == {1, 2, 3}

# Las variables categóricas originales fueron reemplazadas
for columna in variables_categoricas:
    assert columna not in df_transformado.columns

# Las columnas One-Hot contienen únicamente 0 y 1
columnas_onehot = [
    columna
    for columna in df_transformado.columns
    if columna.startswith(
        ("Sexo_", "Zona_", "di3_", "dis2_")
    )
]

for columna in columnas_onehot:
    assert set(
        df_transformado[columna].dropna().unique()
    ).issubset({0, 1})

# Verificación estadística del escalamiento
for columna in variables_numericas:
    valores = df_transformado[columna].dropna()

    assert abs(valores.mean()) < 1e-10
    assert abs(
        valores.std(ddof=0) - 1
    ) < 1e-10

# Dimensión final esperada
assert df_transformado.shape == (5511, 23)
```

```
print("Pipeline integrado validado correctamente.")
print("Dimensiones finales:", df_transformado.shape)
```

Pipeline integrado validado correctamente.

Dimensiones finales: (5511, 23)

Se compara la estructura del resultado con `ens_procesado.csv`, el conjunto de referencia de la Fase 2: dimensiones y columnas faltantes o adicionales.

```
In [28]: # Comparación estructural con el dataset procesado de referencia

ruta_referencia = RAIZ / "data" / "processed" / "ens_procesado.csv"

df_referencia = pd.read_csv(ruta_referencia)

columnas_transformado = set(df_transformado.columns)
columnas_referencia = set(df_referencia.columns)

faltantes = columnas_referencia - columnas_transformado
adicionales = columnas_transformado - columnas_referencia

print("Dimensiones pipeline F3:", df_transformado.shape)
print("Dimensiones referencia:", df_referencia.shape)

print("\nColumnas faltantes:")
print(faltantes)

print("\nColumnas adicionales:")
print(adicionales)

assert df_transformado.shape == df_referencia.shape
assert columnas_transformado == columnas_referencia

print("\nEstructura final compatible con ens_procesado.csv.")
```

Dimensiones pipeline F3: (5511, 23)

Dimensiones referencia: (5511, 23)

Columnas faltantes:

set()

Columnas adicionales:

set()

Estructura final compatible con `ens_procesado.csv`.

Validación estructural del pipeline

Como verificación de integración con el preprocesamiento desarrollado en la Fase 2, se comparó la estructura obtenida mediante los transformadores de la Fase 3 con `ens_procesado.csv`, utilizado como conjunto procesado de referencia.

El pipeline conserva las 5.511 observaciones y genera 23 variables finales. Asimismo, no se detectaron columnas faltantes ni adicionales respecto del conjunto de referencia.

Esta comprobación permite verificar que la refactorización del preprocesamiento mediante componentes modulares mantiene la estructura esperada del dataset, incorporando explícitamente la eliminación de variables no analíticas, el casting de HTA y GPAQ, la codificación One-Hot y el escalamiento de las variables numéricas seleccionadas.

Verificación exacta contra la Fase 2

La comparación anterior solo revisa la estructura (columnas y dimensiones). Para comprobar que también los valores coinciden, se arma el pipeline completo con las clases, partiendo del conjunto sin limpiar, y se compara con el archivo que guardó la Fase 2, celda por celda y considerando los tipos de datos. La tolerancia numérica es de $1e-5$.

```
In [29]: from src.validador import verificar_contra_fase2

pasos_limpieza = [
    MarcadorNoRespuesta("as28"),
    EliminatorFilasNulas("HTA"),
    ImputadorFlexible("IMC", PorMediana()),
    ImputadorFlexible("anos_estudio_MINSAL_1", PorMediana()),
    ImputadorFlexible("as27", PorMedianaDeTramo("as28")),
    ImputadorFlexible("GPAQ", PorModa()),
]

pipeline_completo = Pipeline(pasos_limpieza + pasos_integracion)

coincide = verificar_contra_fase2(
    pipeline_completo,
    datos,
    RAIZ / "data" / "processed" / "ens_procesado.csv",
    verificar_tipos=True,
)
assert coincide
```

[OK] El resultado coincide con la referencia: 5511 filas x 23 columnas comparadas

Resultado de la integración

El pipeline orientado a objetos procesa las 5.511 observaciones y transforma el conjunto inicial de 18 variables en una estructura final de 23 columnas. La estructura obtenida se comparó con ens_procesado.csv, utilizado como referencia del preprocesamiento desarrollado en Fase 2. Ambas versiones presentan dimensiones de 5.511×23 , sin columnas faltantes ni columnas adicionales. La validación final confirma mediante comparación programática que el resultado generado por el pipeline coincide con la referencia en las 5.511 filas y 23 columnas evaluadas. Por lo tanto, la reorganización del preprocesamiento mediante clases y un Pipeline mantiene reproducible el resultado

previamente obtenido, incorporando una arquitectura modular y reutilizable sin alterar el producto final validado.

6.9 Correccion del umbral reportado en la Formativa 3

En la Formativa 3, `construir_indice_dict` armaba el diccionario iterando con `iterrows()`, lo que infla su costo de construccion: cada iteracion crea una Serie de pandas en lugar de operar sobre los valores ya extraidos. Eso llevo a reportar un umbral de 437 busquedas para justificar la construccion del indice, un numero que dependia mas de como se construyo el diccionario que del costo real de indexar.

Se corrigio `construir_indice_dict` (`src/medicion.py`) para usar `dict(zip(...))`. Para aislar exactamente la variable que se corrige, se repite primero la misma comparacion de la Formativa: la busqueda lineal de referencia (`buscar_iterrows_por_clave`) contra la busqueda ya indexada. Como comparacion adicional, se incluye tambien la busqueda vectorizada como linea base, ya que es la alternativa mas rapida disponible sin construir ninguna estructura. La conclusion final del equipo -adoptar `set_index` de pandas como solucion- no depende de ninguna de estas comparaciones y se mantiene sin cambios; lo que se corrige aqui es puntualmente el umbral reportado para la alternativa basada en diccionario.

```
In [30]: from src.medicion import (
        buscar_iterrows_por_clave,
        buscar_vectorizado,
        construir_indice_dict,
        buscar_indexado_dict,
        medir_tiempo_timeit,
    )

datos_indexables = datos.reset_index().rename(columns={"index": "clave_busqueda", "valor_medio": "valor_medio"})
valor_medio = datos_indexables["clave_busqueda"].iloc[len(datos_indexables)-1]

t_construccion = medir_tiempo_timeit(construir_indice_dict, datos_indexables)
indice_dict = construir_indice_dict(datos_indexables, "clave_busqueda")

t_lineal = medir_tiempo_timeit(buscar_iterrows_por_clave, datos_indexables, "clave_busqueda")
t_vectorizada = medir_tiempo_timeit(buscar_vectorizado, datos_indexables, "clave_busqueda")
t_indexada = medir_tiempo_timeit(buscar_indexado_dict, indice_dict, valor_medio)

umbral_vs_iterrows = t_construccion / (t_lineal - t_indexada)
umbral_vs_vectorizada = t_construccion / (t_vectorizada - t_indexada) if t_vectorizada > t_indexada else None

print(f"Construccion del indice (dict con zip): {t_construccion}")
print(f"Busqueda lineal (iterrows, misma referencia que la Formativa 3): {t_lineal}")
print(f"Busqueda vectorizada (alternativa sin iterrows): {t_vectorizada}")
print(f"Busqueda indexada (dict): {t_indexada}")
print()
print(f"Umbral corregido frente a iterrows (misma comparacion que senalo Omar): {umbral_vs_iterrows}")
print(f"Umbral frente a busqueda vectorizada (comparacion adicional): {umbral_vs_vectorizada}")
```

Construcción del índice (dict con zip):	9.4919 ms
Busqueda lineal (iterrows, misma referencia que la Formativa 3):	26.3531 ms
Busqueda vectorizada (alternativa sin iterrows):	0.1537 ms
Busqueda indexada (dict):	0.0751 us

Umbral corregido frente a iterrows (misma comparación que señaló Omar): 0.36 busquedas

Umbral frente a búsqueda vectorizada (comparación adicional): 61.79 busquedas

Frente a la misma línea base que se usó en la Formativa 3 (iterrows()), el umbral corregido queda muy por debajo del valor original de 437 busquedas: conviene construir el diccionario casi desde la primera consulta repetida. Frente a una búsqueda vectorizada -que ya evita el recorrido fila por fila y es mucho más rápida que iterrows()- el punto de equilibrio es más alto, según se ve en la salida de la celda anterior, porque la vectorización reduce buena parte de la ventaja que ofrece indexar. Ambas comparaciones son válidas y responden preguntas distintas: la primera aísla el error señalado sobre la construcción del diccionario; la segunda compara contra la alternativa más rápida disponible sin construir ninguna estructura. En cualquier caso, la conclusión del equipo -adoptar set_index de pandas para las búsquedas repetidas- no depende de ninguna de las dos y se mantiene sin cambios.

7. Patrón de diseño Strategy aplicado a la imputación

Para `as27` había varias formas razonables de rellenar los nulos. Con Strategy, cada forma es una clase con los mismos dos métodos (`calcular` y `rellenar`), e `ImputadorFlexible` la recibe como parámetro. Así se pueden comparar alternativas cambiando solo el objeto que se entrega. Se comparan tres sobre `as27`: media, mediana y mediana por tramo de `as28`.

```
In [31]: estrategias_as27 = [PorMedia(), PorMediana(), PorMedianaDeTramo("as28")]
comparacion = comparar_estrategias(sin_hta, "as27", estrategias_as27)
comparacion
```

```
Out[31]:
```

	estrategia	n_validos	nulos_restantes	media	desv_est	asimetria	cambio_des
0	sin imputar	4518	993	469638.5	624613.8	11.31	
1	media	5511	0	469638.5	565536.7	12.49	-
2	mediana	5511	0	439072.1	569283.3	12.40	-
3	mediana por tramo	4695	816	468979.0	620243.1	11.15	-

La media y la mediana imputan los 993 valores nulos de `as27`, reduciendo la dispersión aproximadamente un 9 %. En cambio, la mediana por tramo imputa 177 casos y

mantiene 816 valores nulos, presentando una variación de solo -0,70 % en la desviación estándar respecto de los datos sin imputar.

Las estrategias responden a criterios diferentes y, por lo tanto, no deben evaluarse únicamente por la cantidad de valores imputados. La mediana por tramo utiliza la información disponible en as28 para realizar la imputación y conserva mejor la dispersión observada en as27. Los 816 casos que permanecen sin imputar se mantienen explícitamente como valores faltantes y deberán considerarse en los análisis posteriores.

La comparación anterior no requirió modificar `ImputadorFlexible`. La celda siguiente lo muestra de forma directa: la misma clase se usa con las tres estrategias y solo cambia el objeto que recibe.

```
In [32]: for estrategia in estrategias_as27:
          paso = ImputadorFlexible("as27", estrategia)
          resultado = paso.ajustar_transformar(sin_hta)
          print(f"{estrategia.etiqueta:<20} nulos restantes en as27: {int(resultado)}")

media                nulos restantes en as27: 0
mediana              nulos restantes en as27: 0
mediana por tramo    nulos restantes en as27: 816
```

Qué habría pasado sin el patrón. En la Fase 2, cada alternativa para `as27` significó una función distinta o una rama `if` dentro de `imputar_nulos_numericos`. Sumar la mediana por tramo habría obligado a modificar esa función y el código que la llama, con el riesgo de alterar la imputación de otras columnas.

Por qué Strategy y no otro patrón. El problema de esta parte es tener varias formas intercambiables de hacer lo mismo sobre una columna. Factory, Observer y Singleton resuelven problemas distintos (decidir qué objeto construir, registrar eventos o compartir una única configuración) que no aparecen en esta etapa del proyecto.

8. Arquitectura y conclusiones

El pipeline construido en las secciones anteriores no es solo una lista de clases que reproducen la Fase 2: es una arquitectura con decisiones de diseño concretas. Esta sección documenta esas decisiones.

8.1 Cohesión y acoplamiento

Cada módulo de `src/` tiene una única responsabilidad: `transformador.py` define el contrato común (`ajustar()`, `transformar()`, `ajustar_transformar()`); `imputadores.py` resuelve exclusivamente el tratamiento de valores faltantes; `transformadores.py` resuelve codificación, escalamiento y las dos transformaciones de limpieza final; `pipeline.py` solo orquesta una secuencia de pasos, sin saber qué

hace cada uno. Esta separación por responsabilidad es alta cohesión: cada módulo agrupa código que cambia por el mismo motivo (McConnell, 2004).

El acoplamiento entre estos módulos es bajo porque la única dependencia real entre ellos es la interfaz de `Transformador`. Esto se puede comprobar directamente en el código, en lugar de solo describirlo:

```
In [33]: from src.imputadores import MarcadorNoRespuesta, EliminatorFilasNulas, ImputadorFlexible, EliminatorColumna, ConvertidorEntero, CodificadorOneHot, EscaladorEstandar, EliminatorColumna
from src.transformadores import CodificadorOneHot, EscaladorEstandar, EliminatorColumna

pasos_del_pipeline = [
    MarcadorNoRespuesta, EliminatorFilasNulas, ImputadorFlexible,
    EliminatorColumna, ConvertidorEntero, CodificadorOneHot, EscaladorEstandar,
    EliminatorColumna
]

for clase in pasos_del_pipeline:
    base = clase.__bases__[0].__name__
    print(f"{clase.__name__}: hereda de {base}")

print()
codigo_pipeline = open(RAIZ / "src" / "pipeline.py").read()
conoce_a_los_pasos = "imputadores" in codigo_pipeline or "transformadores" in codigo_pipeline
print("¿pipeline.py importa alguna de estas clases directamente?", conoce_a_los_pasos)
```

MarcadorNoRespuesta	hereda de Transformador
EliminatorFilasNulas	hereda de Transformador
ImputadorFlexible	hereda de Transformador
EliminatorColumna	hereda de Transformador
ConvertidorEntero	hereda de Transformador
CodificadorOneHot	hereda de Transformador
EscaladorEstandar	hereda de Transformador

¿pipeline.py importa alguna de estas clases directamente? False

Todas las clases heredan de `Transformador`, sin importar en qué módulo estén definidas. `Pipeline` no necesita conocer estos módulos: basta con que cada objeto que recibe cumpla el contrato de la clase base. Este es el mismo principio que sostiene el patrón Strategy usado en la imputación (sección 7): el código que orquesta no conoce los detalles de lo que orquesta (Gamma et al., 1994).

8.2 Flujo de datos del pipeline

El pipeline reproduce secuencialmente las transformaciones definidas para el preprocesamiento de los datos de la ENS:

Entrada: `ens_variables_f1f2.xlsx` → **5.520 filas × 16 columnas**

1. Tratamiento de valores especiales

- `MarcadorNoRespuesta("as28")` : convierte el código `-9999` en valor nulo.

2. Eliminación de observaciones no válidas

- `EliminadorFilasNulas("HTA")` : elimina 9 filas sin diagnóstico válido.

3. Imputación de valores faltantes

- `ImputadorFlexible("IMC", PorMediana())`
- `ImputadorFlexible("anos_estudio_MINSAL_1", PorMediana())`
- `ImputadorFlexible("as27", PorMedianaDeTramo("as28"))`
- `ImputadorFlexible("GPAQ", PorModa())`

4. Eliminación de variables no requeridas

- `EliminadorColumna("IdEncuesta")`
- `EliminadorColumna("FechaInicioF1")`

5. Conversión de tipos

- `ConvertidorEntero("HTA")`
- `ConvertidorEntero("GPAQ")`

6. Codificación One-Hot

- `CodificadorOneHot("Sexo")`
- `CodificadorOneHot("Zona")`
- `CodificadorOneHot("di3")`
- `CodificadorOneHot("dis2")`

7. Escalamiento estándar

- `EscaladorEstandar("Edad")`
- `EscaladorEstandar("IMC")`
- `EscaladorEstandar("as27")`

Salida: `df_transformado` → **5.511 filas × 23 columnas**

Cada paso corresponde a una llamada a `ajustar_transformar()`. El flujo es lineal y unidireccional: no existen ramas condicionales ni pasos que dependan de resultados calculados fuera de su propio proceso de ajuste.

El orden de las transformaciones reproduce la secuencia definida para el preprocesamiento y su resultado final fue verificado en la sección 6.8 contra `ens_procesado.csv`, obteniendo una estructura coincidente de **5.511 filas × 23 columnas**.

8.3 División funcional en lugar de recursividad

El pipeline principal no utiliza recursividad porque las **17 transformaciones que implementan las decisiones de preprocesamiento de Fase 2 siguen una secuencia fija y conocida de antemano**. El problema no presenta una estructura autosimilar ni una profundidad dinámica que justifique llamadas recursivas.

La recursividad resulta apropiada cuando un problema puede expresarse mediante instancias más pequeñas de sí mismo o cuando la profundidad de la estructura no se conoce previamente. Un ejemplo dentro del trabajo desarrollado en Fase 2 es `aplanar()`, utilizada para recorrer metadatos anidados cuya profundidad puede variar.

En el pipeline de Fase 3, implementar recursividad para recorrer los transformadores sería funcionalmente equivalente a una iteración secuencial, pero introduciría llamadas adicionales a la pila de ejecución sin aportar una mejora en claridad, mantenibilidad o eficiencia.

Por esta razón se adopta **división funcional y composición**: cada transformación constituye una unidad independiente representada por una clase, mientras que `Pipeline` coordina su ejecución secuencial. Esta separación permite probar cada componente individualmente, como se evidencia en la sección 3, y sustituir o extender transformaciones sin modificar la lógica general del pipeline.

8.4 Extensibilidad: agregar un paso sin modificar el Pipeline

Un diseño escalable debe permitir requisitos nuevos sin reescribir lo que ya funciona. Se demuestra agregando un paso que el equipo no anticipó al diseñar el pipeline original:

```
In [34]: class RedondeadorDecimales(Transformador):
        """Paso nuevo, no anticipado por el equipo: redondea una columna
        numérica a 2 decimales. Sirve para demostrar que el Pipeline no
        necesita modificarse para aceptar un paso que no existía antes."""

        def aprender(self, df):
            return {}

        def aplicar(self, df):
            df[self.columna] = df[self.columna].round(2)
            return df

pipeline_extendido = Pipeline(pasos_integracion + [RedondeadorDecimales("Eda
resultado_extendido = pipeline_extendido.ajustar_transformar(imputado)

print("Pipeline original: ", df_transformado.shape)
print("Pipeline extendido: ", resultado_extendido.shape)
print("¿Se modificó pipeline.py para lograr esto? No.")
print("¿Se modificó alguna clase existente? No.")
```

```
Pipeline original: (5511, 23)
Pipeline extendido: (5511, 23)
¿Se modificó pipeline.py para lograr esto? No.
¿Se modificó alguna clase existente? No.
```

Pipeline quedó abierto a la extensión, ya que permite incorporar nuevos pasos que cumplan el contrato definido por `Transformador`, y cerrado a la modificación, porque la

incorporación de un nuevo comportamiento no requiere alterar la implementación de Pipeline ni las clases existentes. Esta propiedad se evidencia al agregar RedondeadorDecimales como una nueva subclase de Transformador: el pipeline extendido puede ejecutarla sin modificar pipeline.py ni los componentes previamente implementados. Este comportamiento es consistente con el principio abierto/cerrado (Open/Closed Principle, OCP) del diseño orientado a objetos (Martin, 2017).

8.5 Evolución del proyecto: registro razonado

Fase 2 → Fase 3. En la Fase 2, cada etapa de limpieza se implementaba mediante funciones independientes con ramas `if` para seleccionar distintas alternativas. Por ejemplo, `imputar_nulos_numericos` determinaba internamente qué estrategia de imputación utilizar. En Fase 3, estas operaciones se reorganizaron como clases que comparten la interfaz definida por `Transformador`, mientras que la selección de la estrategia de imputación se desacopló mediante el patrón Strategy (sección 7). De esta forma, se separa la interfaz común de ejecución de cada transformación del comportamiento específico de cada estrategia.

Durante esta entrega. Se identificaron y corrigieron tres aspectos concretos:

1. Los módulos de `src/` dependían de que dicho directorio estuviera agregado directamente a `sys.path`, lo que impedía utilizarlos como un paquete estándar mediante imports como `from src.pipeline import Pipeline` desde la raíz del repositorio. Se corrigieron los imports internos y la configuración de rutas del notebook para permitir el uso consistente de `src/` como paquete.
2. La integración de las etapas de codificación y escalamiento se realizaba encadenando manualmente los transformadores, sin utilizar la clase `Pipeline`. Esta implementación se reemplazó por una instancia real de `Pipeline` (sección 6.8), centralizando la composición y ejecución secuencial de los transformadores. Su resultado fue posteriormente contrastado con `ens_procesado.csv`, obteniendo una estructura coincidente de 5.511 filas x 23 columnas.
3. La medición de `construir_indice_dict` en la Formativa 3 usaba `iterrows()`, lo que inflaba su costo de construcción y distorsionaba el umbral reportado (437 búsquedas). Se corrigió la función para usar `dict(zip(...))`, y se repite la comparación con la misma línea base que usó Omar para detectar el problema (búsqueda con `iterrows`), además de una comparación adicional contra búsqueda vectorizada. Los resultados y el detalle de ambos umbrales se presentan en la sección 6.9; no se repiten aquí para evitar que el texto quede desactualizado si se vuelve a ejecutar el notebook.

Además, se incorporó una sección de validación (sección 3) que ejecuta directamente en el notebook las **59 pruebas de casos normales, límite y excepción** definidas en

`tests/casos_por_clase.py` , en lugar de limitarse a referenciar el archivo que las contiene.

8.6 Conclusiones

El pipeline basado en clases reproduce el resultado de preprocesamiento obtenido en Fase 2: **5.511 filas × 23 columnas**, sin columnas faltantes ni adicionales. La comparación programática realizada mediante `verificar_contra_fase2` en la sección 6.8 confirmó la coincidencia de los valores evaluados respecto de `ens_procesado.csv` . Por lo tanto, la reorganización del código hacia una arquitectura orientada a objetos mantiene el resultado previamente validado, modificando su estructura de implementación y no el producto final del preprocesamiento.

Los principales conceptos de programación orientada a objetos se evidencian mediante código ejecutable: **herencia**, ya que los transformadores concretos derivan de `Transformador` ; **polimorfismo**, porque `Pipeline` utiliza la interfaz común `ajustar_transformar()` independientemente de la clase concreta que recibe; y **encapsulamiento**, mediante la gestión controlada del estado aprendido en `_parametros` . Además, la prueba de extensibilidad de la sección 8.4 demuestra que es posible incorporar nuevos transformadores sin modificar `Pipeline` ni las clases existentes.

La evaluación de eficiencia mostró que no existe una alternativa superior en todas las dimensiones. En las pruebas de mayor tamaño, `EscaladorEstandar` y `CodificadorOneHot` presentaron menores tiempos de ejecución que sus implementaciones de referencia, mientras que `StandardScaler` y `pandas.get_dummies` utilizaron menos memoria pico. Asimismo, para búsquedas repetidas se corrigió el umbral reportado en la Formativa 3 (sección 6.9): frente a la misma línea base que usó Omar (`iterrows`), conviene construir el diccionario casi desde la primera consulta; frente a una búsqueda vectorizada, el punto de equilibrio es más alto. Ambas cifras se calculan en la propia celda de código, no se repiten aquí. Estos resultados permiten fundamentar las decisiones considerando conjuntamente eficiencia temporal, eficiencia espacial e integración arquitectónica.

La principal limitación identificada corresponde a `as27` : después de aplicar la imputación por mediana de tramo permanecen **816 valores nulos**, correspondientes a observaciones para las que la estrategia utilizada no dispone de información suficiente para determinar un valor de reemplazo. Estos casos se conservan explícitamente en lugar de introducir una imputación adicional no definida por la estrategia, por lo que deben considerarse al interpretar análisis posteriores.

Finalmente, la arquitectura desarrollada permite incorporar nuevos pasos de preprocesamiento —por ejemplo, tratamiento de valores atípicos o generación de

variables derivadas— mediante nuevas implementaciones de `Transformador` , sin modificar la lógica general de `Pipeline` .

8.7 Proyección a la Fase 4

Este notebook deja preparado el insumo para la siguiente fase: un conjunto procesado de **5.511 observaciones y 23 variables**, generado mediante un pipeline reproducible cuyos pasos y parámetros pueden inspeccionarse mediante `Pipeline.resumen()` .

Para el análisis de asociaciones de la Fase 4 quedan abiertas tres decisiones metodológicas:

- **Diseño muestral.** Las variables `Fexp_F1F2p_Corr` , `Conglomerado` y `Estrato` se conservan sin transformar para mantener disponible la información asociada al diseño muestral de la ENS. En Fase 4 deberá definirse cómo incorporar estas variables en el análisis estadístico correspondiente.
- **Tratamiento de `as27`** . En esta fase se utilizó escalamiento estándar para `as27` , manteniendo una estrategia consistente con `Edad` e `IMC` . Dada la asimetría observada en la distribución del ingreso, en Fase 4 puede evaluarse una transformación o estrategia de escalamiento alternativa, como `RobustScaler` , verificando previamente su efecto sobre los datos y el análisis posterior. La arquitectura actual permite incorporar esta alternativa mediante un nuevo transformador sin modificar la lógica general de `Pipeline` .
- **Valores faltantes en `as27`** . Después de aplicar la estrategia `PorMedianaDeTramo` , permanecen **816 valores nulos** en `as27` . Estos casos se mantienen identificados mediante las variables auxiliares definidas durante el preprocesamiento, permitiendo decidir explícitamente su tratamiento en los análisis de Fase 4 en lugar de introducir una imputación adicional en esta etapa.

La arquitectura desarrollada permite abordar estas decisiones modificando o incorporando transformadores específicos, sin reescribir el resto del flujo de preprocesamiento.

Fuentes citadas en esta sección

- Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., & Zimmermann, T. (2019). Software engineering for machine learning: A case study. *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 291-300.
<https://doi.org/10.1109/ICSE-SEIP.2019.00042>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- McConnell, S. (2004). *Code complete* (2.a ed.). Microsoft Press.
- scikit-learn developers. (2024). *Pipeline - scikit-learn 1.5 documentation*.
<https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html>